Report 2: Enhanced Dockerized ML Deployment with Web Frontend

U23AI130, U23AI109, U23AI115

April 19, 2026

Contents

1 Introduction **3**

2 Objectives **3**

3 Updated System Architecture **3**

4 Implementation Steps **4**

4.1 Project Directory Structure 4
4.2 Local UI Testing (Initial) 4
4.3 Prediction with Valid Input 5
4.4 Prediction with Different Input 6
4.5 Flask Server Running Locally 7
4.6 Docker Image Build (Multi-Platform) 8
4.7 Pushing Image to AWS ECR 8
4.8 EC2 Deployment and Login 8
4.9 Final Deployment Output 9

5 Results **11**

6 Challenges Faced **12**

6.1 Port Conflict on macOS 12
6.2 Docker Daemon Not Running 12
6.3 Architecture Mismatch (ARM vs AMD64) 12
6.4 ECR Authentication Expiry 12
6.5 Browser Access Issues on Port 5000 12
6.6 Container Startup Delay 12

7 Conclusion **12**

8 Future Work **13**

9 References **13**

1 Introduction

This report presents the enhanced version of the machine learning deployment system developed in Report 1. In this phase, a browser-based frontend interface is integrated with the previously developed backend API. This improvement allows users to interact with the machine learning model through a graphical interface instead of sending manual API requests.

The application is rebuilt, containerized again, and redeployed to AWS EC2 using Docker and AWS Elastic Container Registry (ECR).

2 Objectives

The objectives of this enhanced project are:

- To improve usability by adding a browser-based frontend UI
- To enable user interaction through form-based input
- To maintain backend API functionality alongside UI
- To rebuild and redeploy the updated application using Docker
- To validate the application through browser and API testing

3 Updated System Architecture

The updated system architecture includes:

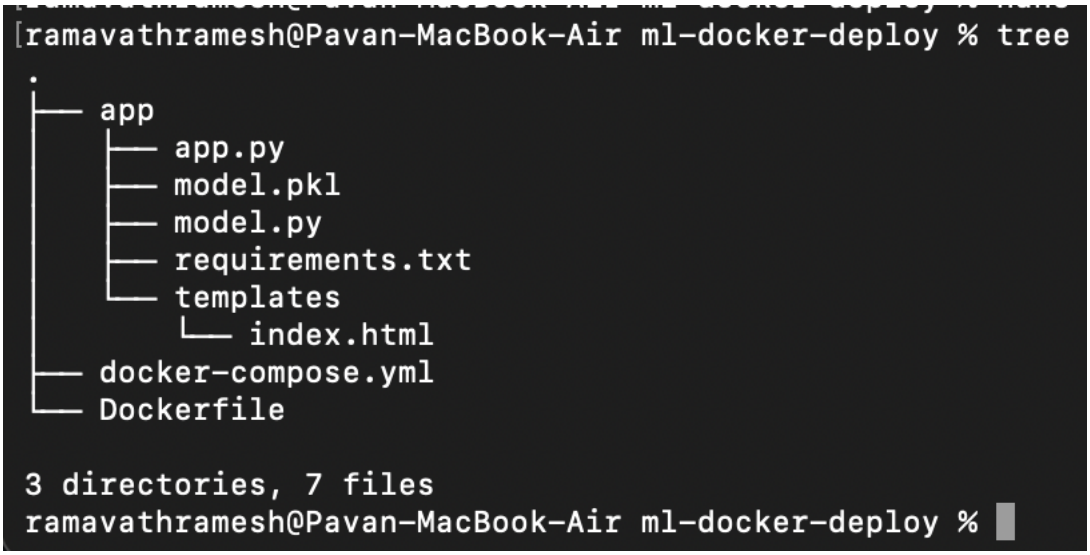
- Machine Learning Model (Random Forest)
- Flask Backend API
- HTML Frontend Interface
- Docker Container
- AWS ECR for image storage
- AWS EC2 for deployment

The key enhancement is the integration of a frontend UI that communicates with the backend.

4 Implementation Steps

4.1 Project Directory Structure

The updated project structure includes a new templates directory for HTML files.



```
ramavathramesh@Pavan-MacBook-Air ml-docker-deploy % tree
.
├── app
│   ├── app.py
│   ├── model.pkl
│   ├── model.py
│   ├── requirements.txt
│   └── templates
│       └── index.html
├── docker-compose.yml
└── Dockerfile

3 directories, 7 files
ramavathramesh@Pavan-MacBook-Air ml-docker-deploy %
```

Figure 1: Updated project directory structure with frontend templates

4.2 Local UI Testing (Initial)

The Flask application was executed locally, and the frontend UI was rendered in the browser.

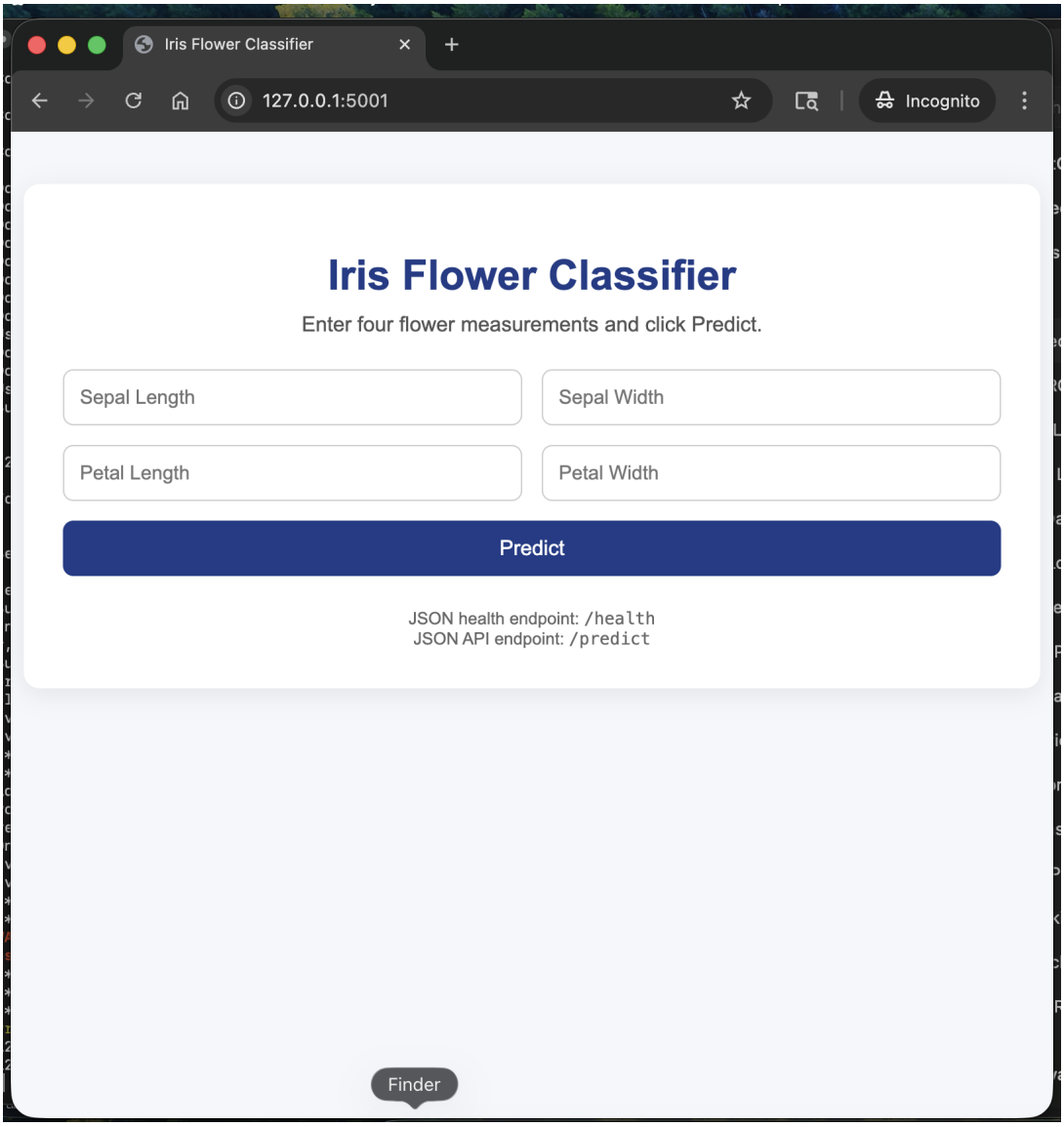


Figure 2: Frontend UI displayed in browser without input values

4.3 Prediction with Valid Input

User input values were entered, and the model successfully predicted the correct class.

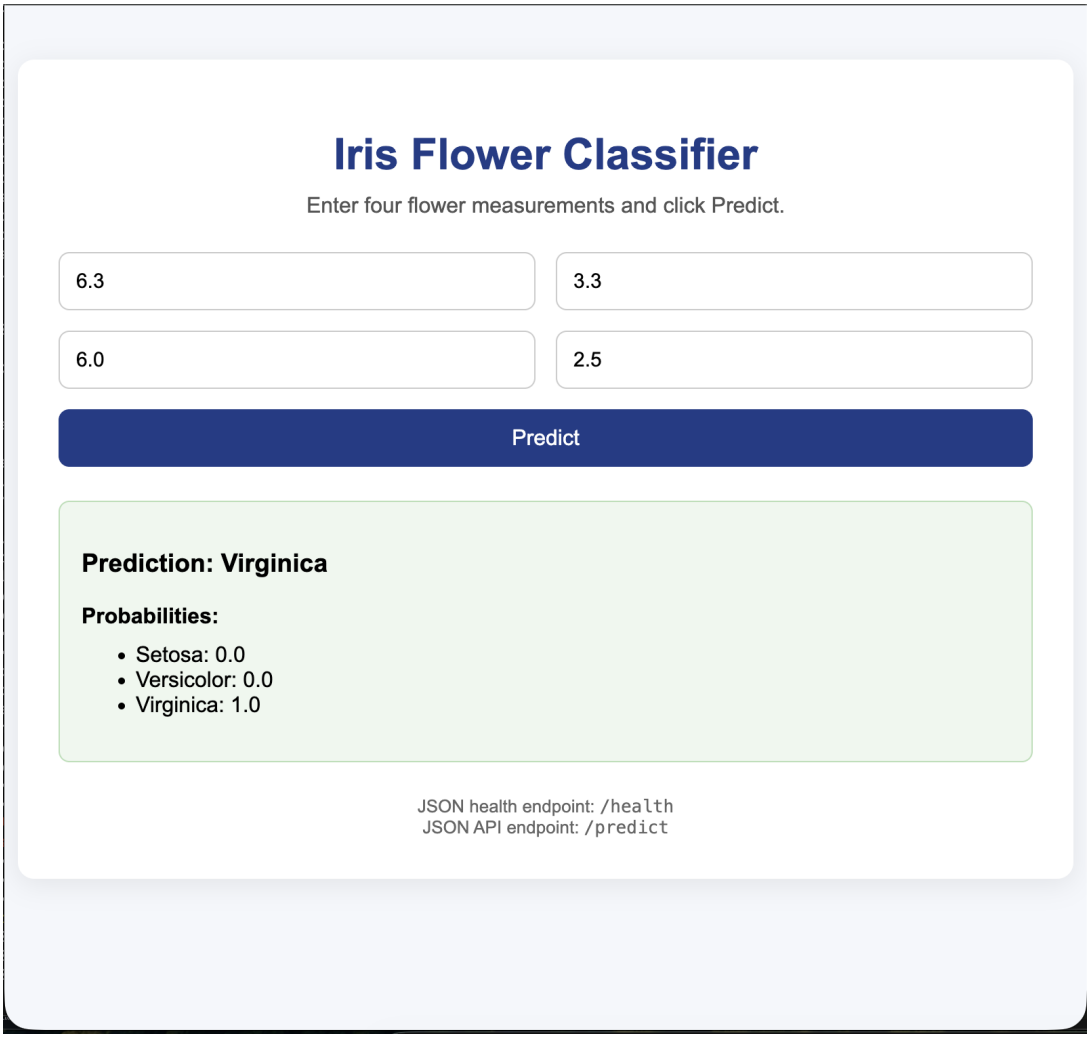


Figure 3: Prediction result for valid Iris input values

4.4 Prediction with Different Input

The system was tested with different input values to validate correctness.

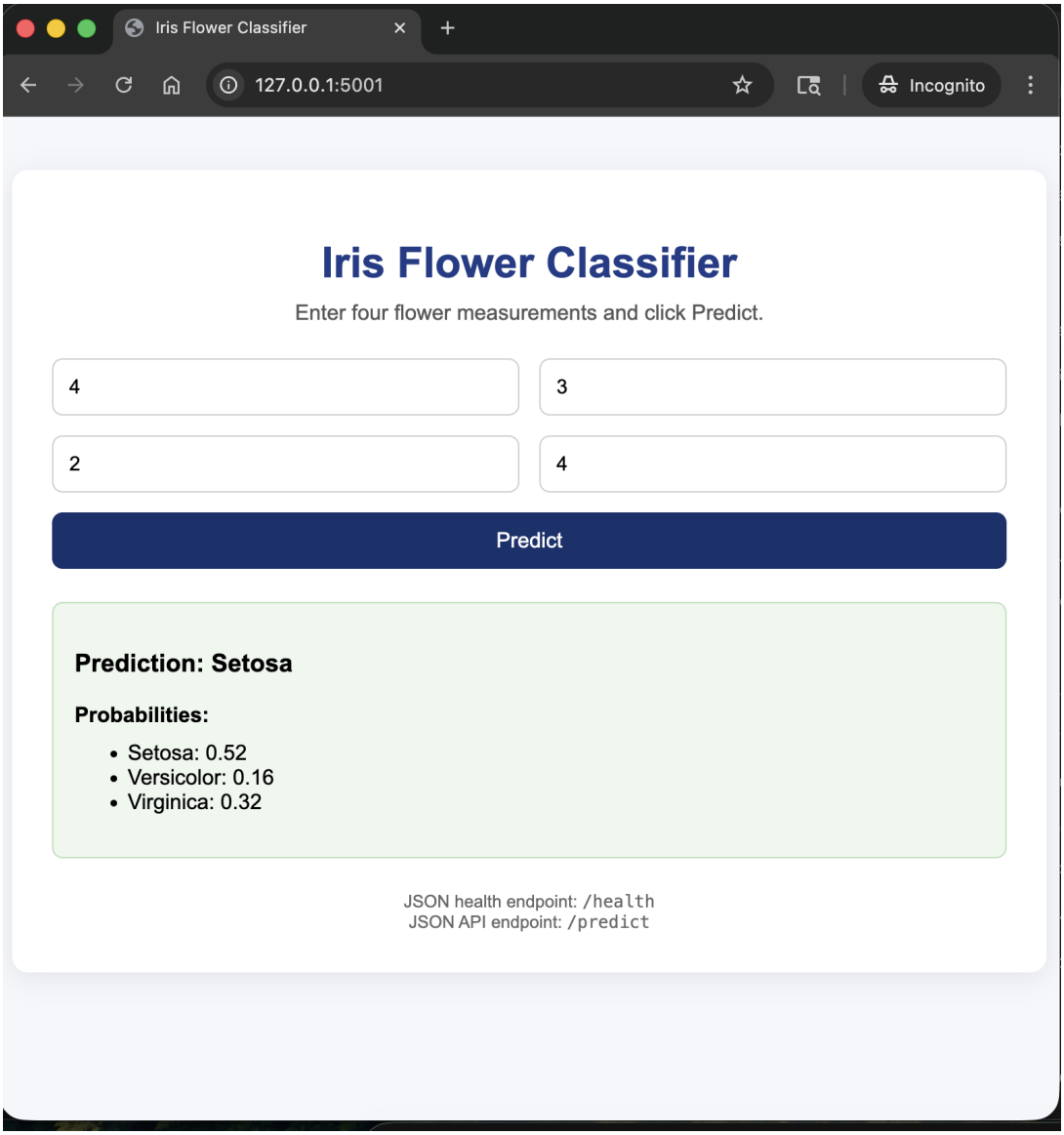


Figure 4: Prediction result for another set of input values

4.5 Flask Server Running Locally

The backend server was verified through terminal output.

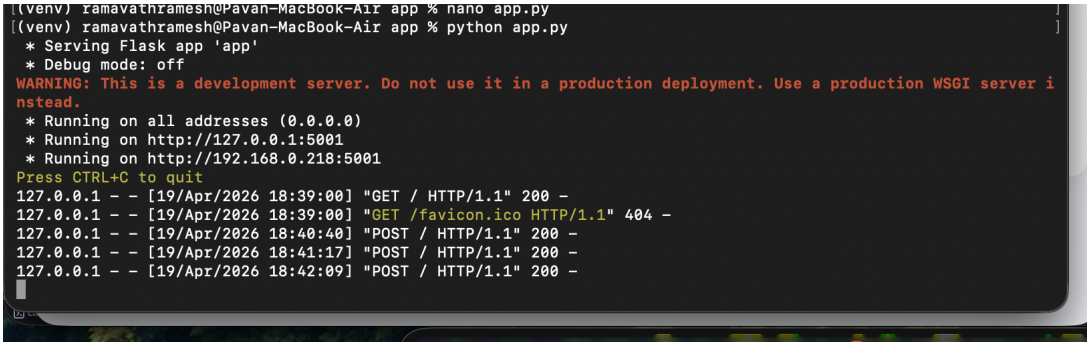


Figure 5: Flask application running locally

4.6 Docker Image Build (Multi-Platform)

The updated application was containerized for Linux AMD64 architecture.

```
(venv) ramavathramesh@Pavan-MacBook-Air ml-docker-deploy % docker buildx build --platform linux/amd64 -t ml-iris-api:latest .
[+] Building 20.8s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.0s
 => => transferring dockerfile: 373B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                  20.7s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 2B                                                                                       0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428              0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c92e73b7566c40428d54d0e79d3428              0.0s
 => [internal] load build context                                                                                     0.0s
 => => transferring context: 6.42kB                                                                                   0.0s
 => CACHED [2/5] WORKDIR /app                                                                                         0.0s
 => CACHED [3/5] COPY app/requirements.txt .                                                                          0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                   0.0s
 => [5/5] COPY app/ .                                                                                                 0.0s
 => exporting to image                                                                                                0.1s
 => => exporting layers                                                                                               0.0s
 => => exporting manifest sha256:719526de5ab17c69b0fee7558f85e4d70c46ed25c7ee01f5ecd36a3b10f70c2d                     0.0s
 => => exporting config sha256:267979dc1a1ac2474f69a1e577dc1d4842c0b64ebc37c5c507a5f9aed0ec6368                       0.0s
 => => exporting attestation manifest sha256:32403046941ff7a1d1b67d841ac2dbb14bb36be9e9e9e00ccc366d52c89              0.0s
 => => exporting manifest list sha256:6e28e09b0c57e05c1f3e7bca7904c29d42ff9af89b0cb8a42b1280318317799b               0.0s
 => => naming to docker.io/library/ml-iris-api:latest                                                                 0.0s
(venv) ramavathramesh@Pavan-MacBook-Air ml-docker-deploy % docker tag ml-iris-api:latest 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
```

Figure 6: Docker build process for updated application

4.7 Pushing Image to AWS ECR

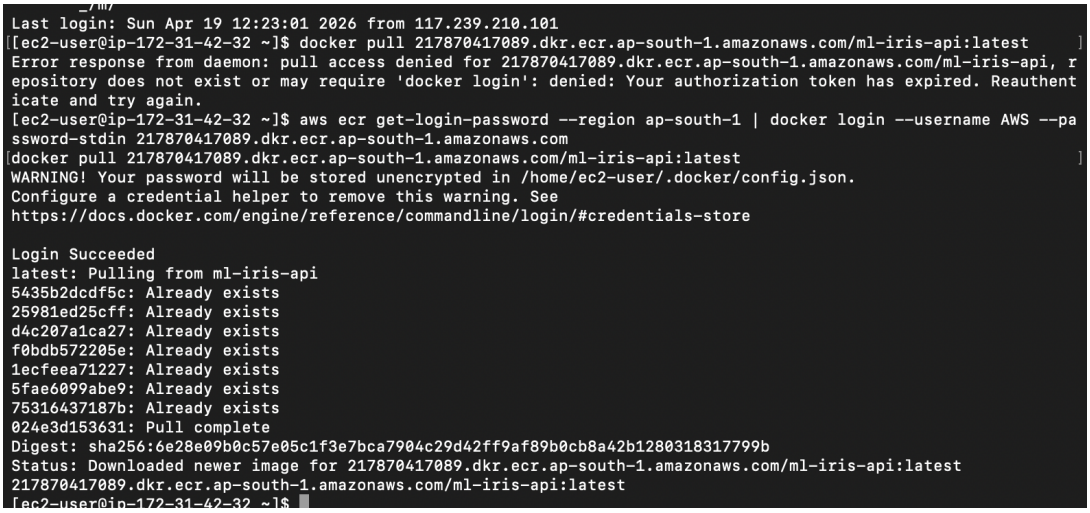
The Docker image was pushed successfully to AWS ECR.

```
ker login --username AWS --password-stdin 217870417089.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
(venv) ramavathramesh@Pavan-MacBook-Air ml-docker-deploy % docker push 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
The push refers to repository [217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api]
5fae6099abe9: Layer already exists
75316437187b: Layer already exists
7a36bac07811: Pushed
d4c207a1ca27: Layer already exists
1ecfeea71227: Layer already exists
5435b2dcdf5c: Layer already exists
f0bdb572205e: Layer already exists
25981ed25cff: Layer already exists
024e3d153631: Pushed
latest: digest: sha256:6e28e09b0c57e05c1f3e7bca7904c29d42ff9af89b0cb8a42b1280318317799b size: 856
(venv) ramavathramesh@Pavan-MacBook-Air ml-docker-deploy %
```

Figure 7: Docker image push to AWS ECR

4.8 EC2 Deployment and Login

The updated image was deployed to EC2, and login was verified.



```
Last login: Sun Apr 19 12:23:01 2026 from 117.239.210.101
[[ec2-user@ip-172-31-42-32 ~]$ docker pull 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
Error response from daemon: pull access denied for 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api, r
epository does not exist or may require 'docker login': denied: Your authorization token has expired. Reauthent
icate and try again.
[ec2-user@ip-172-31-42-32 ~]$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --pa
ssword-stdin 217870417089.dkr.ecr.ap-south-1.amazonaws.com
[docker pull 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
latest: Pulling from ml-iris-api
5435b2dcdf5c: Already exists
25981ed25cff: Already exists
d4c207a1ca27: Already exists
f0bdb572205e: Already exists
1ecfeea71227: Already exists
5fae6099abe9: Already exists
75316437187b: Already exists
024e3d153631: Pull complete
Digest: sha256:6e28e09b0c57e05c1f3e7bca7904c29d42ff9af89b0cb8a42b1280318317799b
Status: Downloaded newer image for 217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
217870417089.dkr.ecr.ap-south-1.amazonaws.com/ml-iris-api:latest
[ec2-user@ip-172-31-42-32 ~]$
```

Figure 8: EC2 login and deployment steps

4.9 Final Deployment Output

The final deployed system was tested using both browser and API endpoints.

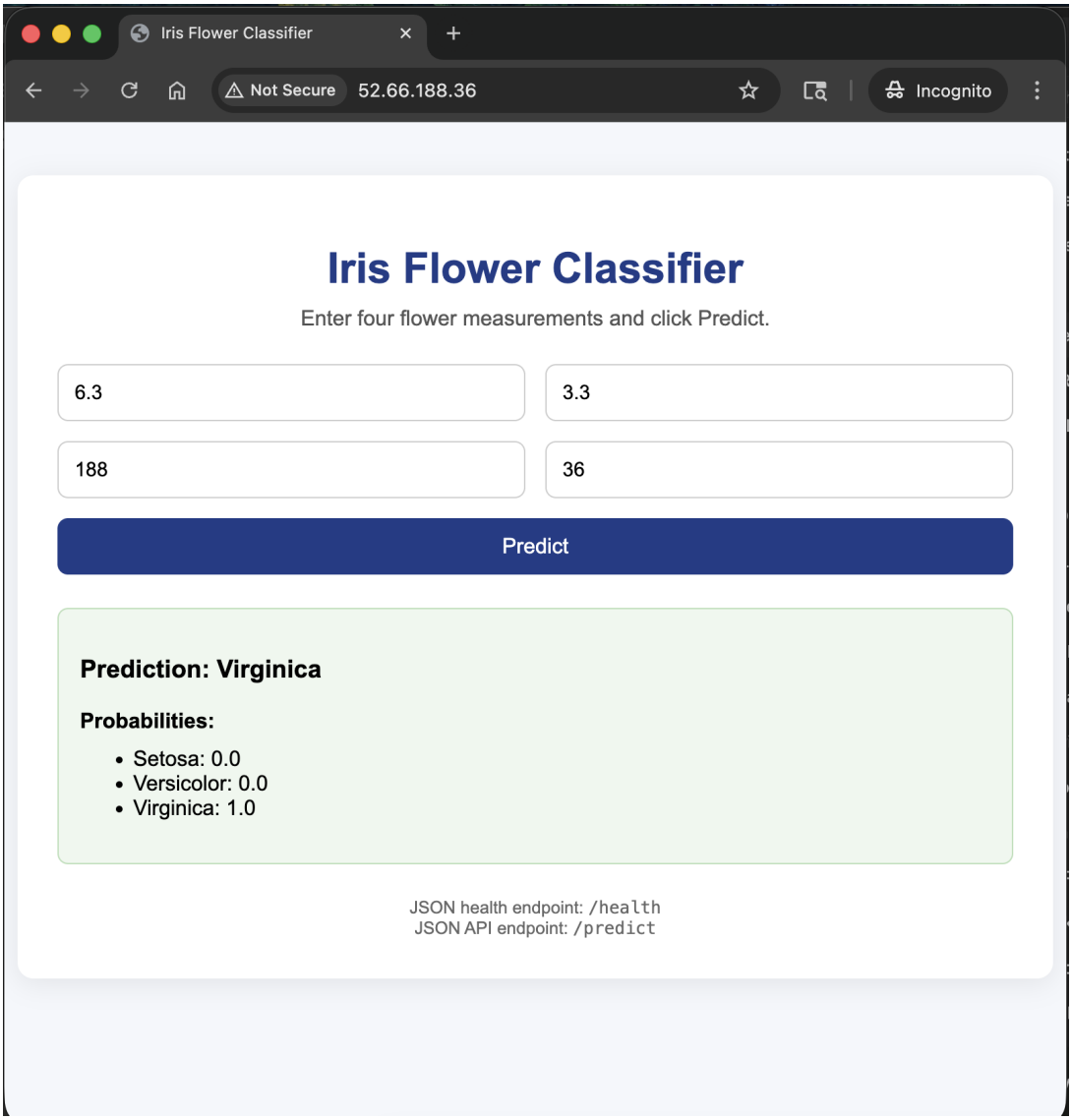


Figure 9: Browser-based frontend working on EC2

```
[[ec2-user@ip-172-31-42-32 ~]$ curl http://localhost/
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Iris Flower Classifier</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f7fb;
            margin: 0;
            padding: 0;
        }

        .container {
            width: 90%;
            max-width: 750px;
            margin: 40px auto;
            background: white;
            border-radius: 12px;
            padding: 30px;
            box-shadow: 0 4px 20px rgba(0,0,0,0.08);
        }

        h1 {
```

Figure 10: API endpoints: HTML root, health check, and prediction response

```
 </body>
[</html>[ec2-user@ip-172-31-42-32 ~]$ curl http://localhost/health
 {"model":"iris-classifier","status":"ok"}
 [ec2-user@ip-172-31-42-32 ~]$
```

Figure 11: API endpoints: HTML root, health check, and prediction response

5 Results

The enhanced system successfully achieved:

- Interactive web-based UI for prediction
- Accurate predictions using ML model
- Successful containerization and deployment
- Public accessibility through EC2 instance
- Functional API endpoints for health and prediction

6 Challenges Faced

6.1 Port Conflict on macOS

Port 5000 was already in use due to system services such as AirPlay. This was resolved by using a different port during local testing.

6.2 Docker Daemon Not Running

Docker commands initially failed because Docker Desktop was not running. This was resolved by starting Docker manually.

6.3 Architecture Mismatch (ARM vs AMD64)

The Docker image built on macOS (ARM) caused compatibility issues with EC2 (AMD64). This was resolved using:

```
docker buildx build --platform linux/amd64
```

6.4 ECR Authentication Expiry

Docker pull failed due to expired ECR token. This was resolved by re-authenticating using AWS CLI.

6.5 Browser Access Issues on Port 5000

Although the API worked using curl, browser access on port 5000 was inconsistent due to client-side behavior. This was resolved by exposing the application on port 80.

6.6 Container Startup Delay

Initial requests failed due to Gunicorn startup delay. Waiting a few seconds resolved the issue.

7 Conclusion

The enhanced system successfully integrates a user-friendly frontend with the machine learning backend. This significantly improves usability and demonstrates a complete real-world deployment pipeline.

The project now includes:

- Machine learning model
- REST API

- Interactive web interface
- Docker containerization
- Cloud deployment on AWS EC2

This implementation reflects a full-stack deployment approach and provides a strong foundation for production-level systems.

8 Future Work

- Add authentication system
- Implement HTTPS using SSL
- Use Nginx as reverse proxy
- Deploy using Kubernetes
- Add logging and monitoring

9 References

- Flask Documentation: `https://flask.palletsprojects.com/`
- Docker Documentation: `https://docs.docker.com/`
- AWS ECR Documentation: `https://docs.aws.amazon.com/ecr/`
- AWS EC2 Documentation: `https://docs.aws.amazon.com/ec2/`
- Scikit-learn Documentation: `https://scikit-learn.org/`